

Programming in Modern C++: Assignment Week 10

Total Marks : 25

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur, Kharagpur – 721302
partha.p.das@gmail.com

September 19, 2025

Question 1

Consider the code segment (in C++11) given below.

[MSQ, Marks 2]

```
#include<iostream>
int main( ){
    double val = 3.5;
    double& ref = val;
    ----- t = ref;    //LINE-1

    t *= 2;
    std::cout << val << " " << ref << " " << t << std::endl;
    return 0;
}
```

Identify the appropriate option/s to fill in the blank at LINE-1 such that output becomes:
7 7 7

- a) `auto`
- b) `auto&`
- c) `decltype(ref)`
- d) `decltype(val)`

Answer: b), c)

Explanation:

In option a), `auto` deduces `t` as `double`, losing the reference. Thus, `t` gets a copy, and modifying `t` does not affect `val`, so output will be 3.5 3.5 7.

In option b), `t` is deduced as `double&`, so it refers to `val`. Updating `t` doubles `val` and `ref`, giving output 7 7 7.

In option c), `decltype(ref)` deduces `t` as the exact type of `ref`, which is `double&`, hence output is 7 7 7.

In option d), `decltype(val)` deduces `t` as `double` (no reference), so `t` gets a copy, and output will be 3.5 3.5 7.

Question 2

Consider the program (in C++11) given below.

[MSQ, Marks 2]

```
#include <iostream>

int main() {
    int a = 10;
    const int b = 20;
    int& r1 = a;
    const int& r2 = b;

    auto x1 = r1;          // x1 -> int (copy)
    auto x2 = b;           // x2 -> int (copy)
    decltype(r2) x3 = b;   // x3 -> const int& (reference to const)
    decltype(a) x4 = a;    // x4 -> int (non-const)

    ++x1;    // LINE-1
    ++x2;    // LINE-2
    ++x3;    // LINE-3
    ++x4;    // LINE-4

    return 0;
}
```

Which of the following line/lines generate/generates a compiler error?

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: c)

Explanation:

- **x1:** deduced by `auto` from `r1` — `auto` drops reference qualifiers, so `x1` is an `int` (modifiable). ****No error**** at LINE-1.
- **x2:** deduced by `auto` from `b` — `auto` drops `const`, so `x2` is an `int` (modifiable). ****No error**** at LINE-2.
- **x3:** declared as `decltype(r2)`, which yields `const int&`. Incrementing a reference to `const int` (LINE-3) is illegal — ****compiler error****.
- **x4:** declared as `decltype(a)`, which is simply `int`, so `x4` is modifiable. ****No error**** at LINE-4.

Thus, only LINE-3 produces a compilation error.

Intentionally kept as MSQ

Question 3

Consider the following code segment.

[MSQ, Marks 2]

```
#include <iostream>

class MyValue{
public:
    constexpr MyValue(int v = 0) : val_{v} { }
private:
    int val_;
};

int runtimeNumber(){
    return 5;
}

constexpr int computeConst(int x){
    return x * 2;
}

int main(){
    int a = 3;
    const int b = 4;

    constexpr MyValue mv1(2);           // LINE-1
    constexpr MyValue mv2(a);           // LINE-2
    constexpr MyValue mv3(runtimeNumber()); // LINE-3
    constexpr MyValue mv4(computeConst(b)); // LINE-4

    return 0;
}
```

Which of the following line/s generate/s a compiler error?

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: b), c)

Explanation:

- LINE-1: The constructor parameter is a literal constant. Object can be **constexpr**.
- LINE-2: The constructor parameter **a** is a non-constant variable, so **constexpr** object cannot be created.
- LINE-3: **runtimeNumber()** is not a **constexpr** function, so its return value cannot be used in **constexpr** initialization.
- LINE-4: **computeConst(b)** is a **constexpr** function with a compile-time constant parameter, so object can be **constexpr**.

Question 4

Consider the following code segment (in C++14).

[MSQ, Marks 2]

```
#include <iostream>
struct Box {
    int value;
    Box(int v) : value(v) {}
    int& get() { std::cout << "Ref "; return value; }
};

struct Container {
    int value;
    Container(int v) : value(v) {}
    int get() { std::cout << "Val "; return value; }
};

template <typename T>
----- { // LINE-1
    return obj.get();
}

int main() {
    Box b{100};
    Container c{200};
    accessor(b) = 500;
    accessor(c);
    return 0;
}
```

Identify the appropriate option/s to fill in the blank at LINE-1 such that the output becomes Ref Val.

- a) `auto accessor(T& obj) -> decltype(obj.get())`
- b) `auto accessor(T& obj)`
- c) `auto& accessor(T& obj)`
- d) `decltype(auto) accessor(T& obj)`

Answer: a), d)

Explanation:

- The call `accessor(b) = 500;` returns an lvalue of type `int&`.
- The call `accessor(c);` returns a prvalue of type `int`.
- Option b) fails since plain `auto` always strips references, so it won't work for the lvalue case.
- Option c) fails since `auto&` always deduces to a reference, which causes issues when returning a prvalue.
- Options a) and d) preserve the correct return type for both cases, making them the correct answers.

Question 5

Consider the following code segment.

[MSQ, Marks 2]

```
#include <iostream>

void display(int* ptr) { /*some code*/ }

template<typename F, typename T>
void invoke(F func, T arg) {
    func(arg);
}

int main() {
    int arr[3] = {1, 2, 3};
    int* q = &arr[0];
    invoke(display, q);           //LINE-1
    invoke(display, 0);           //LINE-2
    invoke(display, NULL);        //LINE-3
    invoke(display, nullptr);     //LINE-4
    return 0;
}
```

Which of the following lines generate/s compiler error?

- a) LINE-1
- b) LINE-2
- c) LINE-3
- d) LINE-4

Answer: b), c)

Explanation:

- For LINE-1, the template type parameter T is deduced to `int*`, so the call is valid.
- For LINE-2, the template type parameter T is deduced to `int`, so `display(int)` causes a compiler error.
- For LINE-3, `NULL` is typically defined as 0 or 0L, so T is deduced to `long int`, leading to a compiler error.
- For LINE-4, `nullptr` has the type `std::nullptr_t`, which is implicitly convertible to any pointer type. Thus, `display(nullptr)` is valid.

Question 6

Consider the following code segment (in C++11).

[MCQ, Marks 2]

```
#include <iostream>
#include <iomanip>

----- { //LINE-1
    return distance * 1000;
}

----- { //LINE-2
    return distance;
}

int main() {
    long double total = 5.0_km + 200.0_m;
    std::cout << "total distance (in meters): " << total;
    return 0;
}
```

Identify the appropriate option to fill in the blanks at LINE-1 and LINE-2 such that the output becomes total distance (in meters): 5200.

- a) LINE-1: long double operator"" km(long double distance)
LINE-2: long double operator"" m(long double distance)
- b) LINE-1: long double operator"" _km(long double distance)
LINE-2: long double operator"" _m(long double distance)
- c) LINE-1: long int operator"" _km(long int distance)
LINE-2: long int operator"" _m(long int distance)
- d) LINE-1: unsigned long long operator"" _km(unsigned long long distance)
LINE-2: unsigned long long operator"" _m(unsigned long long distance)

Answer: b)

Explanation:

- Since the user-defined literals in the program are of type long double, the operators must accept and return long double.
- In C++11, user-defined literals must start with an underscore when using custom suffixes.
- Therefore, option **b)** is the correct answer.

Question 7

Consider the program (in C++11) given below.

[MSQ, Marks 2]

```
#include <iostream>

namespace v1_0 {
    class square{
    public:
        square(int s) : s_(s) {}
        int area(){ return s_ * s_; }
    private:
        int s_;
    };
}

inline namespace v2_0 {
    template<typename T>
    class square{
    public:
        square(T s) : s_(s) {}
        double area(){ return s_ * s_ * 1.0; }
    private:
        T s_;
    };
}

int main(){
    ----- s1(5);           //LINE-1
    ----- s2(5);           //LINE-2
    ----- s3(5.5);         //LINE-3
    std::cout << s1.area() << " " << s2.area() << " " << s3.area();
    return 0;
}
```

Choose the appropriate option to fill in the blanks at LINE-1, LINE-2, and LINE-3 so that the program runs without any compilation error and produces the output:

25 25 30.25

- a) LINE-1: square
LINE-2: square<int>
LINE-3: square<double>
- b) LINE-1: v1_0::square
LINE-2: v2_0::square<int>
LINE-3: square<double>
- c) LINE-1: v1_0::square
LINE-2: square<int>
LINE-3: square<double>
- d) LINE-1: v2_0::square
LINE-2: v2_0::square<int>
LINE-3: v2_0::square<double>

Answer: b), c)

Explanation:

Since `namespace v2_0` is declared as `inline` and it defines the templated version of class `square`, the templated class `square` can be instantiated with or without namespace resolution. However, for `namespace v1_0` (defines the non-templated version of `square`), namespace resolution is mandated.

Question 8

Consider the code segment (C++11) given below.

[MCQ, Marks 2]

```
#include <iostream>
#include <string>
#include <vector>
#include <initializer_list>
using namespace std;

template<class T>
class Data{
public:
    Data() { cout << "default called" << endl; }
    Data(int x) { cout << "single-int called" << endl; }
    Data(double d) { cout << "double called" << endl; }
    Data(initializer_list<int> vals) { cout << "init-list called" << endl; }
    Data(int x, initializer_list<int> vals) {
        cout << "mixed called" << endl;
    }
};

int main(){
    Data<int> d1(5);
    Data<int> d2({1, 2, 3});
    Data<int> d3 = {1, 2, 3};
    Data<int> d4{1, 2, 3};
    Data<int> d5(7, {1, 2, 3});
    Data<int> d6(3.14);
    return 0;
}
```

What will be the output?

- a) single-int called
init-list called
init-list called
init-list called
mixed called
double called
- b) single-int called
init-list called
init-list called
init-list called
mixed called
single-int called
- c) default called
init-list called
init-list called
init-list called
mixed called
double called

d) single-int called
init-list called
init-list called
init-list called
init-list called
double called

Answer: a)

Explanation:

`Data<int> d1(5);` calls the single-parameter constructor `Data(int x){...}`.

`Data<int> d2({1,2,3});`, `Data<int> d3 = {1,2,3};`, and `Data<int> d4{1,2,3};` all invoke the initializer-list constructor `Data(initializer_list<int> vals){...}`.

`Data<int> d5(7,{1,2,3});` calls the mixed constructor

`Data(int x, initializer_list<int> vals){...}`.

Finally, `Data<int> d6(3.14);` matches the `Data(double d){...}` constructor.

Question 9

Consider the C++11 code segment given below.

[MCQ, Marks 2]

```
#include <iostream>

class Obj{
public:
    static int cCreated;
    static int cDeleted;
    Obj(){ ++cCreated; }
    Obj(const Obj& o){ ++cCreated; }
    ~Obj(){ ++cDeleted; }
};

int Obj::cCreated = 0;
int Obj::cDeleted = 0;

class Holder{
private:
    Obj* o;
public:
    Holder(const Obj* _o) : o(new Obj(*_o)){}
    Holder(const Holder& hObj) : o(new Obj(*hObj.o)){}
    Holder& operator=(const Holder& hObj){
        if(this != &hObj){
            delete o;
            o = new Obj(*hObj.o);
        }
        return *this;
    }
    ~Holder(){ delete o; }
};

void swapH(Holder& h1, Holder& h2){
    Holder temp = h1;
    h1 = h2;
    h2 = temp;
}

int main(){
    Obj a;
    Obj b;
    Holder h1(&a);
    Holder h2(&b);
    swapH(h1, h2);
    std::cout << "created = " << Obj::cCreated << ", deleted = " << Obj::cDeleted;
    return 0;
}
```

What will be the output?

- a) created = 4, deleted = 0
- b) created = 3, deleted = 3

c) created = 7, deleted = 3

d) created = 4, deleted = 4

Answer: c)

Explanation:

In main, the statements `Obj a;` and `Obj b;` make the value of `created` = 2, `deleted` = 0 . Then, the statements `Holder h1(&a);` and `Holder h2(&b);` make the value of `created` = 4, `deleted` = 0 .

In `swapH` function, the statement `Holder temp = h1;` makes the value of `created` = 5, `deleted` = 0 . Then, for each of the statements `h1 = h2;` and `h2 = temp;`, the `Obj` object gets deleted first, and then again created by calling the copy constructor of `Obj`. This makes the value of `created` = 7, `deleted` = 3 .

Programming Questions

Question 1

Consider the following program in C++11/14. Fill in the blanks as per the instructions given below:

- at LINE-1 with appropriate header to function `transform`,
- at LINE-2 with appropriate body of function `transform`,

such that it will satisfy the given test cases.

Marks: 3

```
#include <iostream>
#include <string>
#include <algorithm>

int action(int x){
    return x * 2;
}

std::string action(const std::string& s){
    std::string rev = s;
    std::reverse(rev.begin(), rev.end());
    return rev;
}

template <class T>
----- {           // LINE-1
    -----         // LINE-2
}

int main(){
    int a;
    std::string b;
    std::cin >> a >> b;
    std::cout << transform(a) << " ";
    std::cout << transform(b);
    return 0;
}
```

Public 1

Input: 7 code
Output: 14 edoc

Public 2

Input: 5 hello
Output: 10 olleh

Private

Input: 9 cpp
Output: 18 ppc

Answer:

```
LINE-1: auto transform(T n) -> decltype(action(n)) //in C++11
```

```
LINE-1: decltype(auto) transform(T n) //in C++14
```

```
LINE-2: return action(n);
```

Explanation:

The function `transform` must have a trailing return type because its return type depends on the overload of `action()` chosen at compile-time. Thus, the header of `transform` should be:

```
auto transform(T n) -> decltype(action(n)) //in C++11
```

```
decltype(auto) transform(T n) //in C++14
```

The body of the function should be:

```
return action(n);
```

This allows `transform()` to either double an integer or reverse a string depending on the input type.

Question 2

Consider the following program (in C++11).

- Fill in the blank at LINE-1 with appropriate header and initialization list for copy constructor.
- Fill in the blank at LINE-2 with appropriate header for overloading copy assignment operator.
- Fill in the blank at LINE-3 with appropriate header and initialization list for move constructor.
- Fill in the blank at LINE-4 with appropriate header for overloading move assignment operator.

The program must satisfy the sample input and output.

Marks: 3

```
#include <iostream>
```

```
class IntWrap {
public:
    IntWrap() : ip_(nullptr) { }
    IntWrap(int x) : ip_(new int(x)) { }
    ----- {} // LINE-1: copy constructor
    ----- { // LINE-2: copy assignment
        if (this != &ob) {
            delete ip_;
            ip_ = new int(*(ob.ip_) + 10);
        }
        return *this;
    }
    ----- { // LINE-3: move constructor
        ob.ip_ = nullptr;
    }
    ----- { // LINE-4: move assignment
        if (this != &ob) {
            ip_ = ob.ip_;
            ob.ip_ = nullptr;
        }
        return *this;
    }
    void print(){
        if(ip_ == nullptr)
            std::cout << "moved, ";
        else
            std::cout << *ip_ << ", ";
    }
    ~IntWrap(){ delete ip_; }
private:
    int* ip_ = nullptr;
};

int main(){
```

```

    int n;
    std::cin >> n;
    IntWrap w1(n);
    IntWrap w2 = w1;
    IntWrap w3(0);
    w3 = w1;
    w1.print(); w2.print(); w3.print();
    IntWrap w4 = std::move(w1);
    IntWrap w5; w5 = std::move(w1);
    w1.print(); w4.print(); w5.print();
    return 0;
}

```

Public 1

Input: 5

Output: 5, 5, 15, moved, 5, moved,

Public 2

Input: 20

Output: 20, 20, 30, moved, 20, moved,

Private

Input: 7

Output: 7, 7, 17, moved, 7, moved,

Answer:

LINE-1: IntWrap(const IntWrap& ob) : ip_(new int(*(ob.ip_)))

LINE-2: IntWrap& operator=(const IntWrap& ob)

LINE-3: IntWrap(IntWrap&& ob) : ip_(ob.ip_)

LINE-4: IntWrap& operator=(IntWrap&& ob)

Explanation:

At LINE-1, the header and initialization list for copy constructor can be written as:

IntWrap(const IntWrap& ob) : ip_(new int(*(ob.ip_)))

At LINE-2, the header for overloading copy assignment operator can be written as:

IntWrap& operator=(const IntWrap& ob)

At LINE-3, the header and initialization list for move constructor can be written as:

IntWrap(IntWrap&& ob) : ip_(ob.ip_)

At LINE-4, the header for overloading move assignment operator can be written as:

IntWrap& operator=(IntWrap&& ob)

Question 3

Consider the following program (in C++11). Fill in the blanks as per the instructions given below:

- Fill in the blank at LINE-1 with an appropriate template definition.
- Fill in the blank at LINE-2 to complete the header for function multiply.
- Fill in the blank at LINE-3 to define the new type Tmp.

The program must satisfy the sample input and output.

Marks: 3

```
#include <iostream>
class Matrix {
public:
    explicit Matrix(int v = 0) : val(v) {}
    Matrix operator*(int n) {
        return Matrix(val * n);
    }
    Matrix operator*(Matrix m) {
        return Matrix(val * m.val);
    }
    friend std::ostream& operator<<(std::ostream& os, const Matrix& m);
private:
    int val;
};

std::ostream& operator<<(std::ostream& os, const Matrix& m){
    os << m.val;
    return os;
}

----- // LINE-1
----- { // LINE-2
    -----; // LINE-3: define new type Tmp
    Tmp prod = n1 * n2;
    std::cout << prod << std::endl;
}

int main(){
    int a, b, x, y;
    std::cin >> a >> b >> x >> y;
    Matrix m1(x);
    Matrix m2(y);
    multiply(m1, a);
    multiply(m1, m2);
    multiply(a, b);
    return 0;
}
```

Public 1

Input: 2 3 4 5

Output:

8
20
6

Public 2

Input: -1 10 6 -2

Output:

-6
-12
-10

Private

Input: 7 -2 3 4

Output:

21
12
-14

Answer:

LINE-1: `template<typename T, typename U>`

Or

LINE-1: `template<class T, class U>`

LINE-2: `void multiply(T& n1, U& n2)`

LINE-3: `typedef decltype(n1 * n2) Tmp`

Explanation:

At LINE-1, we declare a function template with two generic types.

At LINE-2, we define the function header taking references of two possibly different types.

At LINE-3, we use `decltype` to deduce the result type of the expression `n1 * n2`, and typedef it as `Tmp`.

This allows operator overloading for both `Matrix * int`, `Matrix * Matrix`, and `int * int`.